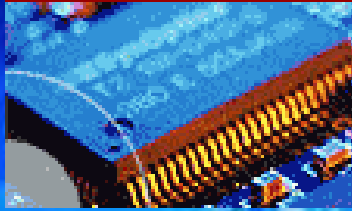
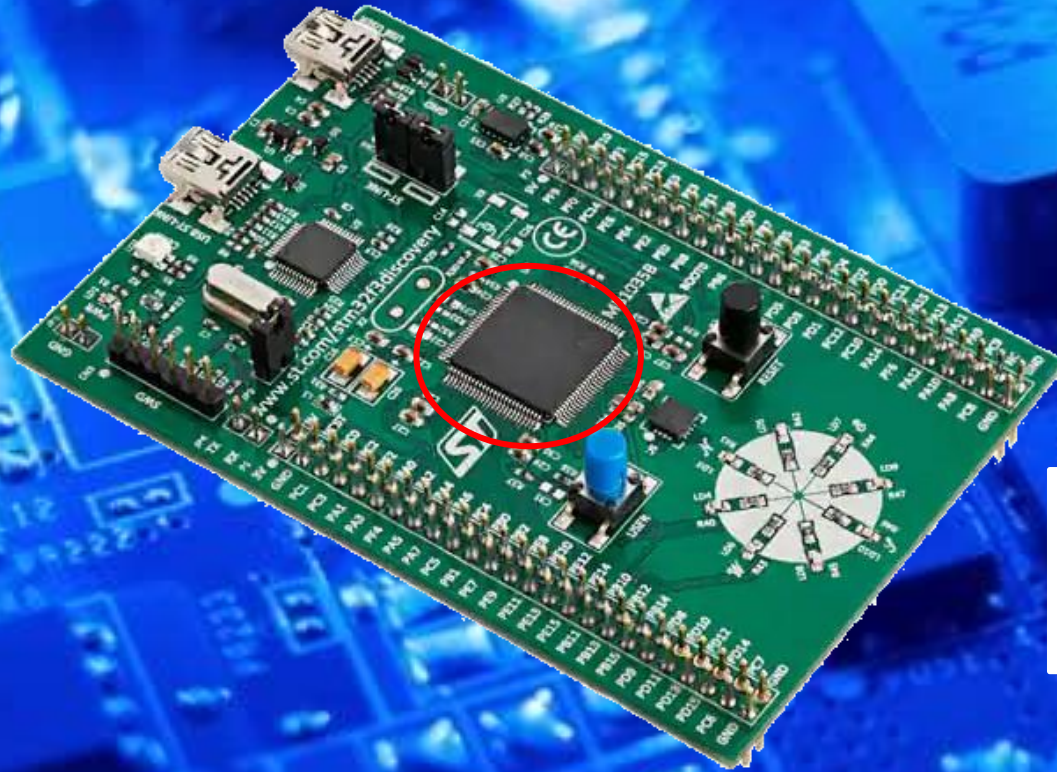


EEE3096S



Embedded Systems II



ARM

Advanced RISC Machines

Part3

ARM Assembly Programming 2/3

Embedded Systems II

L09

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Outline of Lecture

- Recap of ARM instruction types
- ARM Assembly Activity #1
- Activity #1 solution, explanation & simulation

ARM[®]

Advanced RISC Machines

ARM Instruction Set

RECAP

Instruction set divided into six types:

1. Branch Instructions
2. Data Processing instructions
3. Status Register Transfer Instructions
4. Load and Store Instructions
5. Coprocessor Instructions
6. Exception-generating Instructions

ARM Instruction Set

RECAP

Branch

B	Branch
BX	Branch and Exchange
BL	Branch and Link

Data Processing

ADD	Add
SUB	Subtract
RSB	Reverse Subtract
CMP	Compare
TST	Test
AND	Logical AND
EOR	Logical Exclusive OR
MUL	Multiply
SMULL	Sign Long Multiply
SMLAL	Signed Long Multiply Accumulate
MOV	Move
SWP	Swap Word
MVN	Move Not
ADC	Add with Carry
SBC	Subtract with Carry
RSC	Reverse Subtract with Carry
CMN	Compare Negated
TEQ	Test Equivalence
BIC	Bit Clear
ORR	Logical (inclusive) OR
MLA	Multiply Accumulate
UMULL	Unsigned Long Multiply
UMLAL	Unsigned Long Multiply Accumulate
SWPB	Swap Byte

Status-register transfer functions

MSR	Move to Status Register
MRS	Move From Status Register

Load and Store

LDR	Load Word
LDRSH	Load Signed Halfword
LDRSB	Load Signed Byte
LDRH	Load Half Word
LDRB	Load Byte
LDRBT	Load Register Byte with Translation
LDRT	Load Register with Translation
LDM	Load Multiple
STR	Store Word
STRH	Store Half Word
STRB	Store Byte
STRBT	Store Register Byte with Translation
STRT	Store Register with Translation
STM	Store Multiple

Co-Processor

LDC	Load To Coprocessor
CDP	Coprocessor Data Processing
MRC	Move From Coprocessor
STC	Store From Coprocessor

Exception-generating

SWI	Software Interrupt
-----	--------------------

ASL or ASR or integrate as: MOV R1,R2, LSL n where n 0 ... 31

Conditional Execution

RECAP

Suffix

Mnemonic	Condition	CPU condition flags
EQ	Equal	Z set
NE	Not Equal	Z clear
CS	Carry Set/unsigned Higher or Same	C set
CC	Carry Clear/unsigned Lower than	C clear
MI	Negative (Minus)	N set
PL	Positive (Plus)	N clear
VS	oVerflow Set	V set
VC	oVerflow Clear	V clear
HI	Higher unsigned	C set and Z clear
LS	Lower or Same unsigned	C clear or Z set
GE	Greater than or Equal to	(N and V) set or (Nand V) clear
LT	Less Than	(N set and V clear) or (N clear and V set)
GT	Greater Than	((N and V) set or clear) and Z clear
LE	Less Than or equal to	(N set and V clear) or (N clear and V set) or Z set

Addition of Q flag in 64-bit state which indicates floating point saturation

GAS Syntax

RECAP

GAS is the GNU Assembler, which comes with GNU **binutils** for a variety of platforms

Syntax:

[label:] <instruction> <suffix> [operands]

Notes:

- label is optional – only needed if you want to refer to the instruction
- Not all instructions have operands
- Comments start with @

Coding & Thinking Activity



Real ARM Assembly Coding

CPU instructions

- As mentioned in last lecture: this assembly training follows a **two-pass approach**...
- **Pass one:**
 - Dreaming up our own assembly code, thinking what instructions might be useful, how they would look.
- **Pass two:**
 - With a basic understanding of ARM instructions and GAS coding, we now apply this by writing out own assembly code that could run on an ARM

Here is the second pass:

Want to run the same program as before ...

Program Description

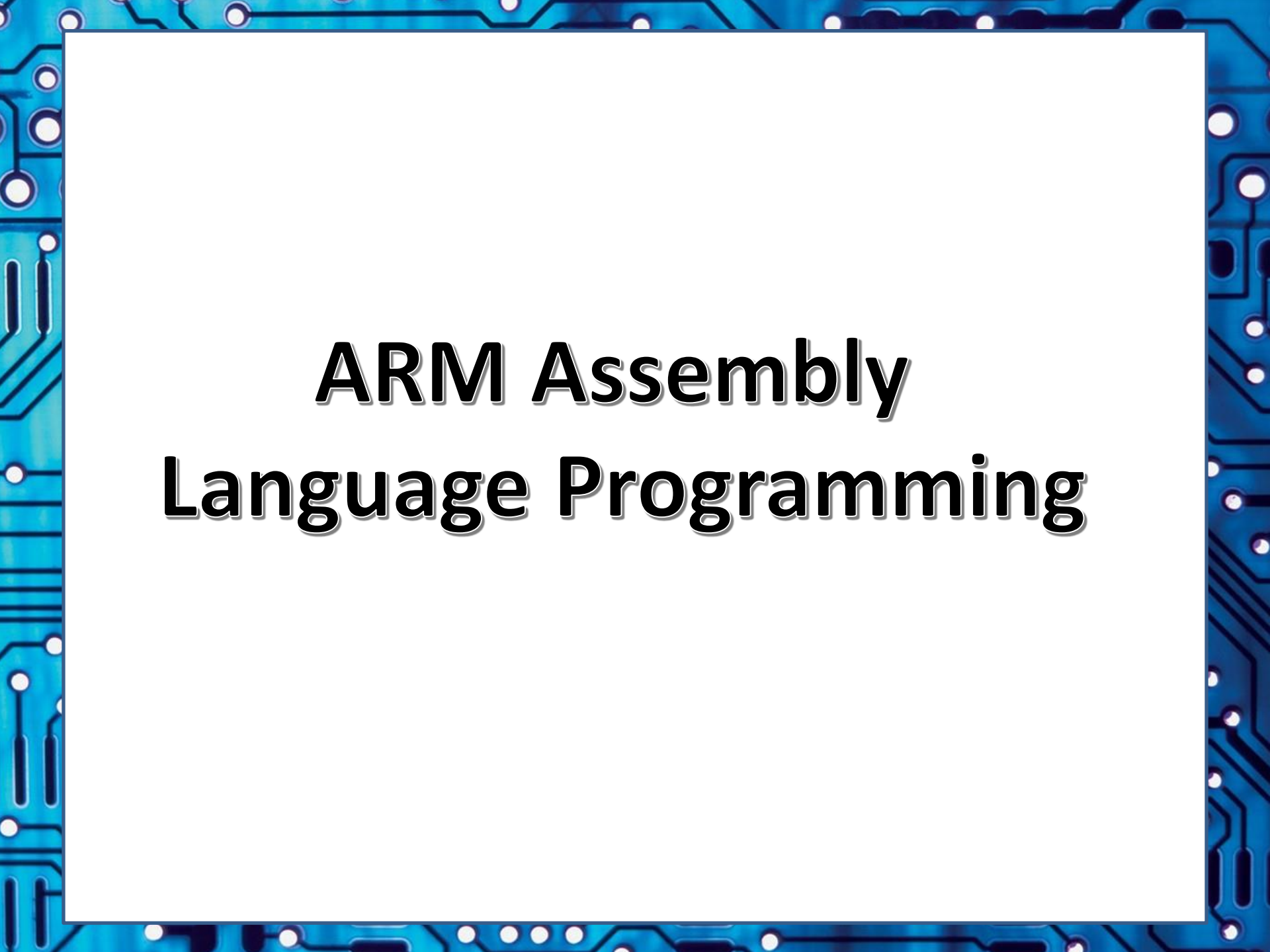
We want to carry out the following processing:

```
int a, b, avg, res;  
a = 100;  
b = 200;  
avg = (a+b) / 2;  
if (a>avg)   res=1;   // a grater than avg  
if (a<avg)   res=-1;  // a less than avg  
if (a==avg)  res=0;   // a same as avg
```

Yes, you can bother to look at that this time!

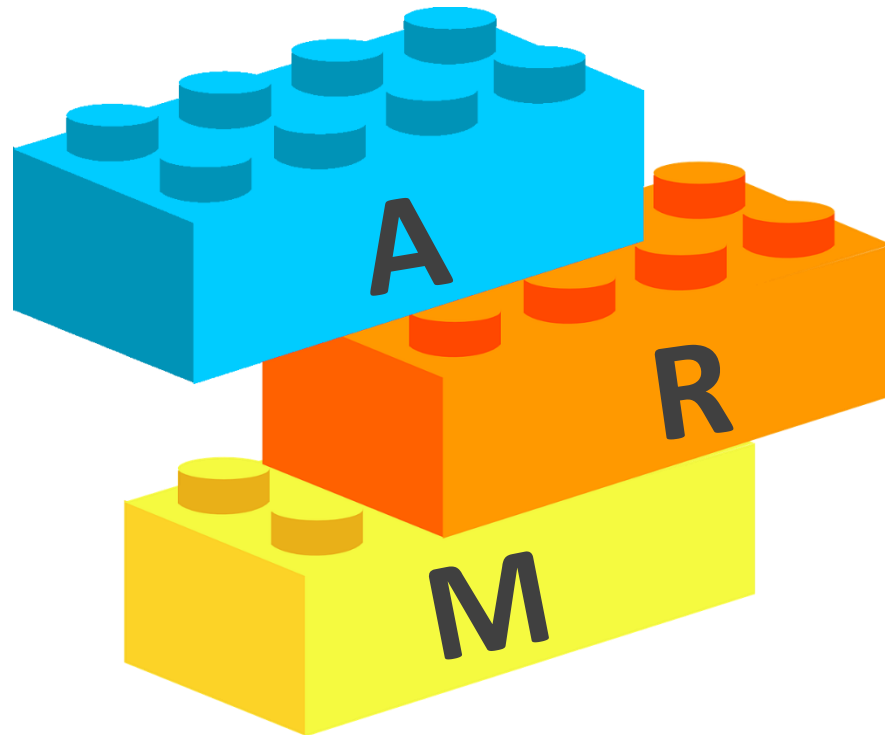


View this C module compaavg.c in the Vula resources (the completed ARM assembly version, called compaavg.s, is included there!)

The image features a blue border with a white circuit board pattern, including lines, circles, and dots, framing a central white rectangle. The text is centered within this white area.

ARM Assembly Language Programming

ARM Assembly Activity #1



(or, if you prefer, Assembly Activity #2)

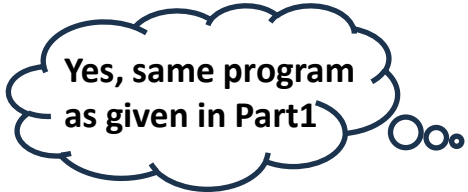
ARM Assembly Activity #1

- For this activity you are tasked to do the simple processing as per the C code on the next slide, but **this time using ARM assembly.**
- *TODO:*
 - Look at the next slide and **contemplate the ARM instructions available**, which have been covered in this lecture, and think **which of them you'd use** to solve this problem.
 - (No need to fully solve this problem yet!)
 - Then proceed to the subsequent slide that gives the solution and explains it.

Activity #1 Problem Description

We want to carry out the following processing:

```
int a, b, avg, res;  
a = 100;  
b = 200;  
avg = (a+b) / 2;  
if (a>avg)   res=1;    // a greater than avg  
if (a<avg)   res=-1;   // a less than avg  
if (a==avg)  res=0;    // a same as avg
```



Yes, same program
as given in Part1

NB: As mentioned before, please think about the instructions covered in these lecture and think which ones you would use where to implement this.

This time use proper ARM instructions, don't make up your own as in the last activity.

View this C module compaavg.c in the Vula resources (the completed assembly version, called compaavg.s, is included there, but don't look at that yet!)

Solution *

RECAP

Program wanted:

```
int a, b, avg, res;  ← 0. Some registers
a = 100;
b = 200;  ← 1. Assign these registers to values
avg = (a+b)/2;  ← 2. Assign avg to (a+b)/2
if (a>avg) res=1;  // a greater than avg  ← 3. Compare & do
if (a<avg) res=-1; // a less than avg      conditional assignment
if (a==avg) res=0;  // a same as avg
```

This given as **reminder** of process to follow in writing assembly, although this case was just pseudocode

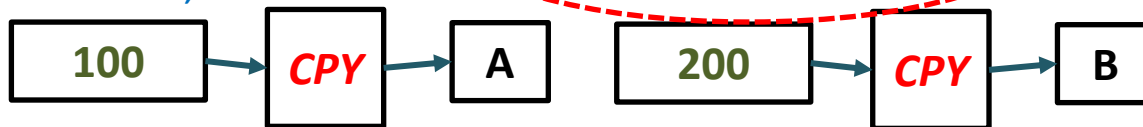
3. Compare & do conditional assignment

0. Some registers, let's say 32-bit ones



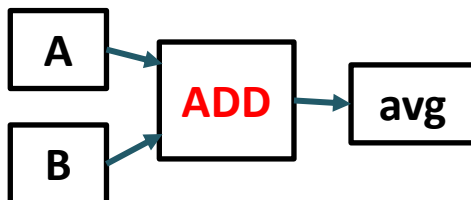
1. CPY A,100
CPY B,200

Decide what real ARM assembly to change to

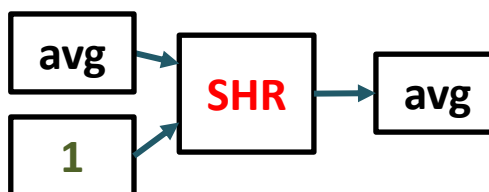


REMINDER This is assembly pseudocode, not real ARM assembly. This done as a quick first attempt to avoid fussing with a specific syntax.

2. ADD AVG,A,B



2. SHR AVG, AVG, 1 // i.e. shift right is /2 for ints



3. CMP ... etc ...

// left as homework ☺

Work with a classmate and try putting together a solution yourselves!

OR: be bored and wait for the sample solution and explanation of how to go about constructing it.



The slide has a decorative border with a blue circuit board pattern, featuring various lines, circles, and dots in a lighter blue color.

ARM Assembly Activity #1

SOLUTION APPROACH & SAMPLE SOLUTION

Activity #1 Solution

Step1: Make a starting point for the procedure you are composing...

```
compaavg:          @ function compaavg that compares  
                   @ param a to average of params a+b
```

Activity #1 Solution

Step2: We need to choose to registers, say r1 and r2 to hold the values for a and b

```
compaavg:                @ function compaavg that compares  
                          @ param a to average of params a+b  
    mov    r1, #100       @ set r1 = a = 100  
    mov    r2, #200       @ set r2 = b = 200
```

Activity #1 Solution

Step3: Now we need to find $\text{avg} = (a+b) / 2$. We are going to do the $/2$ by a arithmetic logic shift right instead of using a divide.

```
compaavg:                @ function compaavg that compares
                           @ param a to average of params a+b
    mov    r1, #100        @ set r1 = a = 100
    mov    r2, #200        @ set r2 = b = 200
    add    r3, r1, r2      @ avg = r3 = a + b
    asr    r3, r3, #1      @ avg = avg / 2
```

Activity #1 Solution

Step4 Now we are going to a comparisons of a to avg, which will set the flags gt (if $a > \text{avg}$) or lt (if $a < \text{avg}$) or eq (if $a = \text{avg}$). However, we only need two branches because the last option doesn't need a branch.

```
compaavg:                @ function compaavg that compares
                           @ param a to average of params a+b
mov    r1, #100           @ set r1 = a = 100
mov    r2, #200           @ set r2 = b = 200
add    r3, r1, r2         @ avg = r3 = a + b
asr    r3, r3, #1         @ avg = avg / 2
cmp     r1, r3            @ compare r1 to r3, i.e. a vs. avg
bgt     Greater           @ branch to Greater if a>avg
blt     Lesser            @ branch to Lesser if a<avg
Equal:                    @ can put a label for this default,
                           @ execution proceed here if a==avg
```

Now we need to implement the default (if none of the branches are taken) as well as the body of the branches...

Activity #1 Solution

Step5 handling of `a==avg`.

```
compaavg:          @ function compaavg that compares
...
    cmp    r1, r3    @ compare r1 to r3, i.e. a vs. avg
    bgt    Greater   @ branch to Greater if a>avg
    blt    Lesser    @ branch to Lesser if a<avg
    Equal:          @ can put a label for other option,
                    @ execution proceed here if a==avg
    mov    r0, #0    @ set ret = 0 and return
    b      Return    @ branches to return*
```

That's the default... now let's do the body of the branches

* Could simply return at this point, but I'm trying to make the code easier to follow, so am leaving just a jump to the end of this function.

Activity #1 Solution

Step6 handling of a>avg and a<avg branch bodies

```
compaavg:                @ function compaavg that compares
...
    cmp    r1, r3        @ compare r1 to r3, i.e. a vs. avg
    bgt    Greater       @ branch to Greater if a>avg
    blt    Lesser        @ branch to Lesser if a<avg
Equal:                    @ can put a label for other option,
                          @ execution proceed here if a==avg

    mov     r0, #0        @ set ret = 0 and return
    b      Return         @ branches to return

Greater:
    mov     r0, #1        @ set ret = 0 and return
    b      Return

Lesser:
    mov     r0, #0        @ a bit more complicated because -1 is 0xFFFFFFFF
    sub     r0, r0, #1    @ so is more that 12 bits available in an instruction
    b      Return         @ therefore I'm just subtracting 1 from 0 to get -1.
```

Activity #1 Solution

Step7 Finally doing the return.

```
compaavg:                @ function compaavg that compares
...
    cmp    r1, r3        @ compare r1 to r3, i.e. a vs. avg
    bgt    Greater       @ branch to Greater if a>avg
    blt    Lesser        @ branch to Lesser if a<avg
...
Lesser:
    mov     r0, #0        @ a bit more complicated because -1 is 0xFFFFFFFF
    sub     r0, r0, #1    @ so is more that 12 bits available in an instruction
    b       Return        @ therefore I'm just subtracting 1 from 0 to get -1.
Return:
    bx lr               @ standard returns from function
                        @ the rule for C is that the return value is
                        @ stored in r0. Which is what we have set up.
```

Try it online at <https://wunkolo.github.io/OakSim/> see next slide ...

OakSim: Online ARM Simulator

OakSim is a very useful ARM AArch32 simulator that is used to run the example.

See: <https://wunkolo.github.io/OakSim/>

It has simple functionality but is well suited to starting out in learning ARM assembly coding. The usual file extension for GCC or NASM ARM assembly is .s. The previous slides explained how to create this file.

When starting OakSim it puts in the The following example code, a while (1) loop.

The screenshot shows the OakSim web interface. At the top, the title "OakSim" is displayed. Below it is a control bar with buttons: "Open", "Run", "250", "Step", and "Reset". The "Step" button is highlighted with a yellow arrow and the text "Controls to run, step or reset the program". To the left of the control bar is a text area for writing assembly code, containing a loop example, with a yellow arrow pointing to it and the text "Write your assembly code here". To the right of the control bar is a "Register Value" table with registers R0 through R7, all showing a value of 0. A yellow arrow points to this table with the text "View registers here". Below the register table is a memory dump showing hexadecimal addresses and values. A yellow arrow points to this section with the text "View machine code and memory contents here".

OakSim

Open Run 250 Step Reset

```
1 Loop:
2   add r0, r0, #1
3   mul r1, r0, r0
4   b Loop
```

Register Value

R0	0
R1	0
R2	0
R3	0
R4	0
R5	0
R6	0
R7	0

0x00010000: 01 00 80 E2 90 00 01 E0 FC FF FF EA 00 00 00 00
0x00010010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010030: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010110: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010130: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010140: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010150: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010160: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

© 2017

Testing compaavg.s on OakSim

Select all the text in the left window (the assembly code) and delete it, then copy and past the text contents of compaavg.s into that window. Press Reset. Then step through the code to see how the registers change. You should eventually get R0 = - 1

OakSim

Step though the code

Open Run 250 Step Reset

```
1 @ EEE3096S 2020 Assmble Programming Class Activity 1
2
3 compaavg:      @ function compaavg that compares
4                @ param a to average of params a+b
5  mov  r1, #100 @ set r1 = a = 100
6  mov  r2, #200 @ set r2 = b = 200
7  add  r3, r1, r2 @ avg = r3 = a + b
8  asr  r3, r3, #1 @ avg = avg / 2
9  cmp  r1, r3    @ compare r1 to r3, i.e. a vs. avg
10 bgt  Greater   @ branch to Greater if a>avg
11 blt  Lesser    @ branch to Lesser if a<avg
12 Equal:        @ can put a label for other option,
13                @ execution proceed here if a==avg
14  mov  r0, #0    @ set ret = 0 and return
15  b     Return
16
17 Greater:
18  mov  r0, #1    @ set ret = 0 and return
19  b     Return
20 Lesser:
21  mov  r0, #0    @ a bit more complicated because
22  sub  r0, r0, #1 @ so that takes more that 16 bits
23  b     Return   @ therefore I'm just subtracting
```

Register Value

R0	-1
R1	64
R2	c8
R3	96
R4	0
R5	0
R6	0
R7	0

See how registers change

0x00010000: 64 10 A0 E3 C8 20 A0 E3 02 30 81 E0 C3 30
0x00010010: 03 00 51 E1 02 00 00 CA 03 00 00 BA 00 00
0x00010020: 04 00 00 EA 01 00 A0 E3 02 00 00 EA 00 00
0x00010030: 01 00 40 E2 FF FF FF EA 1E FF 2F E1 00 00
0x00010040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x000100F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010110: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010130: 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00010140: 00 00 00 00 00 00 00 00 00 00 00 00 00 00

© 2017

Assembly Activity Done



Second pass of learning assembly:
writing an ARM assembly program
using real ARM instructions.



Hopefully you now know the specific
approach now of writing ARM assembly
that can be combined by GAS.

Of course you might get something along these lines in a test or exam. But moreover the important aspect is knowing a bit about instructions that are commonly available on a CPU and how they work.

Concluding remarks

- This is a brief starting point to ARM coding, we haven't yet looked at calling convention for assembly routines, the passing of parameters, using the stack, etc. etc.
- Lecture 11 clarifies many of these issues explaining how to mix C and assembly code, because, nowadays, it is seldom that you will ever write a program entirely in assembly!

The Next Episode...

Lecture L10

- Using ARM Assembly Programming 3/3:
 - interfacing C and ARM assembly
 - program startup, runtime library, linking

